National University of Computer and Emerging Sciences

AL-2002: Artificial Intelligence Lab
Lab Exercise 09

Department of Computer Science
FAST-NU, Lahore, Pakistan

Instructions:

- Write all code in Python using scikit-learn unless stated otherwise.
- Use random_state=42 for all train/test splits unless specified.
- Add comments in your code where asked. Marks are awarded for both correctness and explanation.
- Bonus questions are optional but highly recommended.

PART A — Classification Tasks

Q1	Breast Cancer Binary Classifier		

Using sklearn's built-in load_breast_cancer() dataset, build a complete classification pipeline from scratch.

TASKS:

- Load the dataset and print the shape of features and labels.
- Split into 80% train and 20% test using random_state=42.
- Scale features using StandardScaler.
- Train a LogisticRegression model.
- Print the accuracy score on the test set.

STARTER IMPORTS:

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
```

Q2	Compare Two Classifiers		

Using the Iris dataset (same as the lab manual), train two different classifiers on the same train/test split and compare their performance.

TASKS:

- Train a LogisticRegression model and record its accuracy.
- Train a KNeighborsClassifier with k=3 and record its accuracy.
- Print both accuracies side by side in a formatted output.
- Print the confusion matrix for the better-performing model.
- In a comment, state which classifier performed better and give a one-line reason.

> **Hint:** Use the same X_train, X_test, y_train, y_test for both models so the comparison is fair. Wrap each model in its own variable (e.g. lr_model, knn_model).

Q3	Fix the Buggy Pipeline		

The following code has three deliberate bugs. Identify and fix each one, then run it and print the correct accuracy.

BUGGY CODE:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

iris = load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test  = scaler.fit_transform(X_test)   # Bug 1

model = LogisticRegression()
model.fit(X_test, y_test)               # Bug 2

y_pred = model.predict(X_train)         # Bug 3
print("Accuracy:", accuracy_score(y_test, y_pred))
```

TASKS:

- Identify and describe each bug in a comment above the corrected line.
- Write the complete corrected version of the script and run it.

PART B — Regression Tasks

Q4	California Housing — Linear Regression		

Replicate the regression workflow from the lab manual using the California Housing dataset.

TASKS:

- Load fetch_california_housing() and print the dataset shape.
- Split 80/20 with random_state=42 and scale features using StandardScaler.
- Train a LinearRegression model.
- Print MSE, RMSE, and R^2 score.
- In a comment, explain what $R^2 = 0.57$ means in plain English.

> **Hint:** RMSE = np.sqrt(mean_squared_error(y_test, y_pred)). R^2 close to 1.0 means the model explains most of the variance in the target.

Q5	Compare Linear vs Decision Tree Regressor		

Train both regressors on the California Housing data and compare them using a structured printed output.

TASKS:

- Train LinearRegression and DecisionTreeRegressor(random_state=42) on the same split.
- For each model print: MSE, RMSE, and R^2 score.
- Print a conclusion: which model performed better and why (based on metrics).

EXPECTED OUTPUT FORMAT:

```
--- Linear Regression ---
MSE:  0.5559
RMSE: 0.7456
R²:   0.5758

--- Decision Tree ---
MSE:  0.4938
RMSE: 0.7027
R²:   0.6230

Better model: ...
```

Q6	Effect of Test Size on Performance		

Investigate how varying the train/test split ratio affects model performance.

TASKS:

- Use the California Housing dataset with LinearRegression.
- Run the full pipeline (scale → train → evaluate) for test_size in [0.1, 0.2, 0.3, 0.4, 0.5].
- For each split, record the R^2 score.
- Print a table showing test_size vs R^2 for all five runs.
- Write a 2-sentence observation about the trend you see.

> **Hint:** Use a for loop: for size in [0.1, 0.2, 0.3, 0.4, 0.5]: and repeat the split → scale → fit → score steps inside. Keep random_state=42 for reproducibility.

PART C — Model Evaluation

Q7	Read the Confusion Matrix		

Given the confusion matrix output from the Iris classification example in the lab manual:

```
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

TASKS:

- How many samples were in the test set in total?
- How many predictions were correct?
- How many predictions were wrong? Name the class(es) that had errors.
- Compute the accuracy by hand and show the formula used.
- Write code using sklearn to generate and print this same matrix from scratch using the Iris dataset.

Q8	Classification Report Analysis		

Train a KNeighborsClassifier(n_neighbors=7) on the breast cancer dataset and produce a full evaluation report.

TASKS:

- Print the accuracy score.
- Print the full classification_report.
- From the report, identify which class has the lowest F1-score.
- Answer in a comment: what does F1-score measure and why is it useful beyond accuracy?

> **Hint:** F1-score = 2 × (precision × recall) / (precision + recall). It is especially useful when classes are imbalanced — accuracy alone can be misleading in such cases.

Q9	Scaling vs No Scaling		

Investigate the effect of feature scaling on KNN classification performance.

TASKS:

- Load the breast cancer dataset and split 80/20 with random_state=42.
- Train KNeighborsClassifier(n_neighbors=5) without any scaling — record accuracy.

- Train the same classifier with StandardScaler applied — record accuracy.
- Print both accuracies and the difference between them.
- In comments, explain why scaling matters specifically for KNN (relate to how KNN computes distance).

BONUS — Challenge Questions (Optional)

These questions are optional. Full marks are awarded for correct, well-commented code.

QB1	KNN Hyperparameter Sweep		

Find the optimal value of k for KNN classification on the Iris dataset.

TASKS:

- Loop over k = 1, 3, 5, 7, 9, 11 using the same train/test split.
- For each k, train a KNeighborsClassifier and record test accuracy.
- Print a formatted table: k | accuracy.
- State the best k value and explain in a comment why very small k (k=1) can be problematic.

> **Hint:** k=1 means the model memorizes training data perfectly and is very sensitive to noise — this is called overfitting.

QB2	Build a Reusable ML Pipeline Function		

Write a reusable Python function that automates the entire ML classification pipeline and prints a formatted summary report.

FUNCTION SIGNATURE:

```
def run_classifier(model, X, y, test_size=0.2, scale=True):
    """
    Splits data, optionally scales features, trains the model,
    and prints accuracy + confusion matrix.

    Parameters:
        model      : any sklearn-compatible classifier
        X          : feature matrix
        y          : target labels
        test_size  : fraction of data for testing (default 0.2)
        scale      : whether to apply StandardScaler (default True)
    """
    # your implementation here
    pass
```

TASKS:

- Implement the function with all parameters as described above.
- Call it with LogisticRegression on the Iris dataset.
- Call it with KNeighborsClassifier(n_neighbors=5) on the breast cancer dataset.
- The function must work for any sklearn-compatible classifier passed in.